# Microservices Architecture & Design Patterns

Ultimate Interview Preparation Cheat Sheet for Spring Boot Ecosystem

| Pattern | Where Used in Your Code | Primary Business Goal |
|---|---|---|
| API Gateway | **GatewayAPI** (Port 8383) | Single entry point, handles centralized JWT token validation & header injection. |
| Service Discovery | **EurekaServer** (Port 8761) | Dynamically tracks instance IPs to allow service-to-service communication. |
| Centralized Config | **ConfigServer** (Port 8181) | Decouples active configuration properties from local code repos. |
| Circuit Breaker | **Resilience4j** in Report & Notification | Blocks cascading microservice crashes by triggering fallback actions. |
| Saga Pattern | Site / Booking & Notification services | Orchestrates transactions cleanly across independent domain schemas. |

## 1. API Gateway Pattern (Centralized Router & Auth Filter)

**Where Used:** GatewayAPI microservice (Port 8383)
**Key Classes/Config:** AuthenticationGatewayFilterFactory.java & JwtUtil.java

**How it works:** Acts as the unified front door for all client requests. When the client sends an HTTP request, the Gateway intercepts it, decrypts and validates the state of the JWT token, extracts client identity (Id and Roles), and propagates this identity downstream by adding **'X-User-Id'** and **'X-User-Roles'** headers.

**■■ How to explain in the Interview:**

*"We implemented the **API Gateway Pattern** using Spring Cloud Gateway to establish a unified gateway entry point. It decouples security checks from business microservices by verifying stateless JWT tokens centrally, extracting crucial metadata headers, and routing traffic dynamically to downstream microservices."*

## 2. Service Discovery Pattern (Netflix Eureka Server)

**Where Used:** EurekaServer (Port 8761) & all 10 registered services
**Key Classes/Config:** @EnableEurekaServer (server) & eureka.client.service-url.defaultZone (clients)

**How it works:** Enables microservices to discover and invoke one another without hardcoding physical network IP addresses or host ports. Every microservice registers its location on startup, and Feign client calls are routed based on service names.

**■■ How to explain in the Interview:**

*"We use the **Service Discovery Pattern** via **Netflix Eureka**. Services dynamically register their metadata on start. Downstream Feign interfaces resolve endpoints dynamically using lookup names rather than statically bound configurations, enabling elastic scaling."*

## 3. Centralized External Configuration Pattern

**Where Used:** ConfigServer (Port 8181) loading shared profiles
**Key Classes/Config:** spring.config.import=optional:configserver:http://localhost:8181

**How it works:** Separates configuration environments entirely from compiled binary code. This allows database urls, Resilience4j properties, security settings, and credentials to be adjusted centrally without rebuilding the microservices.

**■■ How to explain in the Interview:**

*"We implemented the **Externalized Central Configuration Pattern** with Spring Cloud Config Server. Shared configurations are decoupled from microservice repositories, enabling centralized management, secret control, and hot property reloading."*

## 4. Circuit Breaker Pattern (Microservice Fault Tolerance)

**Where Used:** ReportingService (Port 9090) & NotificationService (Port 7070)
**Key Classes/Config:** application.properties (slidingWindowSize, failureRateThreshold, waitDurationInOpenState)

**How it works:** Prevents catastrophic failure cascades across services. If a microservice call fails repeatedly, the circuit trips OPEN, immediately skipping network attempts and calling custom local fallback routines.

**■■ How to explain in the Interview:**

*"We protect system stability using the **Circuit Breaker Pattern** via **Resilience4j**. If calls to a downstream service reach a 50% failure rate over 10 attempts, the circuit breaks, instantly returning fallback structures (like mock objects) to protect callers."*

## 5. Database-per-Service Pattern (True Domain Isolation)

**Where Used:** Throughout all 11 microservice schemas
**Key Classes/Config:** Independent schemas: reportdb, notificationdb, compliancedb, etc.

**How it works:** Ensures that each microservice possesses complete, unshared ownership over its own relational schema. Direct cross-database access is prohibited; all interaction occurs strictly via logical service interfaces.

**■■ How to explain in the Interview:**

*"To maintain strict decoupling and allow autonomous development, we use the **Database-per-Service Pattern**. Each service has its own dedicated schema, preventing database coupling and enforcing clean boundary division."*

## 6. Saga Pattern (Distributed Transactions / Event-Choreography)

**Where Used:** Site / Booking services communicating with NotificationService
**Key Classes/Config:** Event/REST notifications triggered during site and event creation

**How it works:** Coordinates multi-step transaction workflows across separate service databases. Instead of blocking databases, each microservice executes a local transaction, sending a request to the next service to trigger sequential actions.

**■■ How to explain in the Interview:**

*"Since distributed databases prevent two-phase locks, we coordinate cross-service business transactions using the **Saga Pattern (Choreography-based)**. Services execute their local database changes and trigger downstream events via REST APIs."*